

Your understanding is mostly on the right track. Below is a corrected and production-oriented version of your notes, with a few important distinctions about FutureTask, wildcards, thread execution, and the CompletableFuture methods.

## Java Future and CompletableFuture

### 1. Submitting tasks to an ExecutorService

ExecutorService.submit() has three commonly used overloads:

```
1 Future<?> submit(Runnable task);
2
3 <T> Future<T> submit(Runnable task, T result);
4
5 <T> Future<T> submit(Callable<T> task);
```

The method you choose determines:

- Whether the task returns a value
  - What type of Future you receive
  - What Future.get() returns
- 

### 2. Runnable versus Callable

#### Runnable

A Runnable:

- Does not return a value
- Cannot directly throw checked exceptions
- Defines a run() method

```
1 Runnable task = () -> {
2     System.out.println("Executing task");
3 };
```

Its functional method is:

```
1 void run();
```

#### Callable<T>

A Callable<T>:

- Returns a value of type T
- Can throw checked exceptions

- Defines a call() method

```
1 Callable<Integer> task = () -> {  
2     return 100;  
3 };
```

Its functional method is:

```
1 T call() throws Exception;
```

## 3. The three forms of submit()

### 3.1 submit(Runnable)

```
1 ExecutorService executor = Executors.newFixedThreadPool(2);  
2  
3 Future<?> future = executor.submit(() -> {  
4     System.out.println("Running task");  
5 });
```

Because the Runnable does not return a result:

```
1 Object result = future.get();  
2 System.out.println(result); // null
```

The future represents the task's completion, cancellation, or failure, but the successful result is always null.

---

### 3.2 submit(Runnable, T result)

```
1 Future<String> future = executor.submit(  
2     () -> System.out.println("Processing"),  
3     "COMPLETED"  
4 );  
5  
6 String result = future.get();  
7 System.out.println(result); // COMPLETED
```

The Runnable itself does not calculate or return "COMPLETED".

The executor returns the predefined value after the Runnable finishes successfully.

Conceptually:

```
1 runTheRunnable();  
2 return predefinedResult;
```

---

### 3.3 submit(Callable<T>)

```
1 Future<Integer> future = executor.submit(() -> {
2     return 10 + 20;
3 });
4
5 Integer result = future.get();
6 System.out.println(result); // 30
```

Here, the result is calculated by the task itself.

---

## 4. What happens internally during submit()?

A simplified mental model is:

```
1 Runnable or Callable
2     ↓
3 Wrapped in a FutureTask
4     ↓
5 FutureTask submitted to ThreadPoolExecutor
6     ↓
7 Worker thread executes FutureTask.run()
8     ↓
9 FutureTask records result, exception, or cancellation
10    ↓
11 Waiting threads are notified
```

For example, an implementation can conceptually perform:

```
1 RunnableFuture<T> futureTask = new FutureTask<>(callable);
2 execute(futureTask);
3 return futureTask;
```

### Important correction

It is not quite accurate to say:

▮ The thread pool updates the FutureTask state based on the thread's state.

The worker thread executes FutureTask.run(), but FutureTask **manages its own lifecycle state**.

Conceptually, its states include:

```
1 NEW
2   ↓
3 COMPLETING
4   ↓
5 NORMAL or EXCEPTIONAL
```

Cancellation can lead to states conceptually representing:

```
1 CANCELLED
2 INTERRUPTING
3 INTERRUPTED
```

The thread's lifecycle and the FutureTask's lifecycle are different.

For example:

- A worker thread may remain alive after completing the task.
  - The FutureTask may be completed while the worker thread starts another task.
  - Cancelling the future does not necessarily mean the executor's worker thread is permanently terminated.
- 

## 5. Relationship between Future and FutureTask

Future is an interface:

```
1 public interface Future<V>
```

It provides methods such as:

```
1 boolean cancel(boolean mayInterruptIfRunning);
2 boolean isCancelled();
3 boolean isDone();
4 V get();
5 V get(long timeout, TimeUnit unit);
```

FutureTask<V> is a concrete class that implements both Runnable and Future<V>:

```
1 public class FutureTask<V>
2     implements RunnableFuture<V>
```

RunnableFuture<V> extends both:

```
1 Runnable
2 Future<V>
```

Therefore, a FutureTask has two important roles:

1. **Executable task**, because it is a Runnable
2. **Result and state holder**, because it is a Future

A useful mental model is:

```
1 FutureTask = executable computation + completion state +  
result/exception
```

## 6. What does Future<?> mean?

The ? is an **unbounded wildcard**.

```
1 Future<?>
```

It means:

A Future containing some type, but the exact type is unknown or irrelevant at this point.

For example:

```
1 Future<?> future = executor.submit(() -> {  
2     System.out.println("No result");  
3 });
```

Because submit(Runnable) represents a task without a useful result, its return type is Future<?>.

When calling get():

```
1 Object result = future.get();
```

The only type-safe assumption Java can make is that the result is an Object. For submit(Runnable), the successful result will specifically be null.

### Future<?> does not mean any value can be assigned

This is valid:

```
1 Future<String> stringFuture = executor.submit(() -> "hello");  
2 Future<?> unknownFuture = stringFuture;
```

But once it is stored as Future<?>, the specific result type is no longer known:

```
1 Object value = unknownFuture.get();
```

This is not type-safe:

```
1 String value = unknownFuture.get(); // Compilation error
```

A cast would be required, though it should only be used if the type is genuinely known:

```
1 String value = (String) unknownFuture.get();
```

## 7. Limitations of Future

Future is useful for representing a pending computation, but it has limited composition capabilities.

For example:

```
1 Future<Integer> future = executor.submit(() -> 100);  
2  
3 Integer result = future.get();
```

get() is a blocking call. The calling thread waits until:

- The task completes
- The task fails
- The task is cancelled
- A specified timeout expires

A plain Future does not naturally support:

- Chaining transformations
- Combining multiple asynchronous computations
- Registering completion callbacks
- Declarative exception handling
- Building asynchronous pipelines

This is where CompletableFuture becomes useful.

---

## 8. CompletableFuture

CompletableFuture<T> implements two important interfaces:

```
1 Future<T>  
2 CompletionStage<T>
```

The Future<T> side gives it completion-related operations such as get().

The CompletionStage<T> side gives it asynchronous composition operations such as:

- thenApply
- thenCompose
- thenCombine
- thenAccept
- exceptionally

It can also be completed manually:

```
1 CompletableFuture<String> future = new CompletableFuture<>();
2
3 future.complete("SUCCESS");
```

## 9. supplyAsync()

supplyAsync() starts an asynchronous computation that returns a value.

```
1 CompletableFuture<Integer> future =
2     CompletableFuture.supplyAsync(() -> 100);
```

It accepts a Supplier<T>:

```
1 @FunctionalInterface
2 public interface Supplier<T> {
3     T get();
4 }
```

### Without an explicit executor

```
1 CompletableFuture<Integer> future =
2     CompletableFuture.supplyAsync(() -> 100);
```

It normally uses ForkJoinPool.commonPool().

### With an explicit executor

```
1 ExecutorService executor = Executors.newFixedThreadPool(4);
2
3 CompletableFuture<Integer> future =
4     CompletableFuture.supplyAsync(
5         () -> 100,
6         executor
7     );
```

In production code, passing an explicit executor is often preferable because it gives you control over:

- Pool size
- Thread naming
- Queueing behavior
- Workload isolation
- Shutdown
- Monitoring
- Rejection policy

You should avoid putting long-running blocking I/O operations into the common fork-join pool because unrelated asynchronous operations may share that pool.

---

## 10. runAsync()

If the asynchronous operation does not return a value, use runAsync().

```
1 CompletableFuture<Void> future =
2     CompletableFuture.runAsync(() -> {
3         System.out.println("Performing background work");
4     });
```

It accepts a Runnable and returns:

```
1 CompletableFuture<Void>
```

A simple distinction is:

```
1 runAsync()    → Runnable → no result
2 supplyAsync() → Supplier → produces a result
```

## 11. thenApply() versus thenApplyAsync()

Both methods transform the result of the previous stage.

```
1 CompletableFuture<Integer> future =
2     CompletableFuture.supplyAsync(() -> 10)
3         .thenApply(value -> value * 2);
```

The result is:

```
1 10 → 20
```

The function passed to thenApply receives an input and returns an output:

```
1 Function<T, U>
```

Conceptually:

```
1 T → U
```

### thenApply()

```
1 .thenApply(value -> value * 2)
```

The non-async version may execute:

- In the thread completing the previous stage, or
- In the calling thread if the previous stage has already completed



Therefore, you should not rely on `thenApply()` always using the exact same thread as `supplyAsync()`.

## `thenApplyAsync()`

```
1 .thenApplyAsync(value -> value * 2)
```

This schedules the transformation for asynchronous execution.

Without an explicit executor, it normally uses the default asynchronous executor.

With an explicit executor:

```
1 .thenApplyAsync(value -> value * 2, executor)
```

## Important correction

`thenApplyAsync()` does **not guarantee a newly created thread**.

It means the work is submitted for asynchronous execution. An existing worker thread from the relevant pool will usually execute it.

A better distinction is:

```
1 thenApply      → continuation may execute inline
2 thenApplyAsync → continuation is scheduled asynchronously
```

## 12. `thenCompose()` versus `thenApply()`

`thenCompose()` is used when the transformation itself starts another asynchronous operation.

Consider:

```
1 CompletableFuture<User> findUser(int userId)
```

and:

```
1 CompletableFuture<List<Order>> findOrders(User user)
```

You can compose them:

```
1 CompletableFuture<List<Order>> ordersFuture =
2     findUser(10)
3     .thenCompose(user -> findOrders(user));
```

Execution dependency:

```
1 Find user
2   ↓
3 Use that user to find orders
```

```
4      ↓  
5  Produce List<Order>
```

The generic signature is conceptually:

```
1  <U> CompletableFuture<U> thenCompose(  
2      Function<? super T, ? extends CompletionStage<U>> function  
3  );
```

The function:

- Accepts the previous result, or one of its supertypes
- Returns a CompletionStage<U> or one of its subtypes

## Why not use thenApply()?

Using thenApply() would produce a nested future:

```
1  CompletableFuture<CompletableFuture<List<Order>>> nested =  
2      findUser(10)  
3      .thenApply(user -> findOrders(user));
```

Using thenCompose() flattens it:

```
1  CompletableFuture<List<Order>> flattened =  
2      findUser(10)  
3      .thenCompose(user -> findOrders(user));
```

This is similar to the difference between:

```
1  map      → thenApply  
2  flatMap  → thenCompose
```

## About ordering

thenCompose() maintains **dependency ordering**, not necessarily thread ordering.

Task 2 cannot begin through this chain until Task 1 produces the value required by Task 2. However, Task 1 and Task 2 may execute on different threads.

---

## 13. Understanding ? super T and ? extends CompletionStage<U>

Consider:

```
1  Function<? super T, ? extends CompletionStage<U>>
```

## ? super T

The function may accept:

- T
- A superclass of T
- An implemented superinterface of T

For example, if T is Integer, a function accepting Number can also process it.

```
1 Function<Number, CompletableFuture<String>> function =  
2     number ->  
CompletableFuture.completedFuture(number.toString());
```

## ? extends CompletionStage<U>

The function may return:

- CompletionStage<U>
- CompletableFuture<U>
- Another subtype of CompletionStage<U>

A useful rule is:

```
1 Consumer/input position → ? super T  
2 Producer/output position → ? extends T
```

This is often remembered as PECS:

```
1 Producer Extends, Consumer Super
```

## 14. thenAccept() and thenAcceptAsync()

thenAccept() consumes the previous result without returning another business value.

```
1 CompletableFuture<Void> future =  
2     CompletableFuture.supplyAsync(() -> "Order created")  
3     .thenAccept(message -> System.out.println(message));
```

It accepts a:

```
1 Consumer<T>
```

A Consumer<T> receives a value and returns nothing:

```
1 void accept(T value);
```

Therefore, thenAccept() returns:

```
1 CompletableFuture<Void>
```

Is it necessarily the end of the chain?

Not technically.

Because it returns a `CompletableFuture<Void>`, more stages can still be attached:

```
1 CompletableFuture<Void> future =  
2     CompletableFuture.supplyAsync(() -> "Order created")  
3         .thenAccept(System.out::println)  
4         .thenRun(() -> System.out.println("Notification  
completed"));
```

So it is better to say:

`thenAccept()` is a value-consuming stage, commonly used near the end of a pipeline.

The `async` execution distinction is the same:

```
1 thenAccept      → may execute inline  
2 thenAcceptAsync → scheduled asynchronously
```

## 15. `thenRun()` and `thenRunAsync()`

`thenRun()` is useful when the next action does not need the previous result.

```
1 CompletableFuture<Void> future =  
2     CompletableFuture.supplyAsync(() -> "Result")  
3         .thenRun(() -> System.out.println("Task completed"));
```

Difference:

```
1 thenAccept(value -> ...) → receives the previous result  
2 thenRun(() -> ...)      → does not receive the previous result
```

## 16. `thenCombine()` and `thenCombineAsync()`

`thenCombine()` combines the results of two independent completion stages.

```
1 CompletableFuture<String> userFuture =  
2     CompletableFuture.supplyAsync(() -> "Suhaib");  
3  
4 CompletableFuture<Integer> scoreFuture =  
5     CompletableFuture.supplyAsync(() -> 95);  
6  
7 CompletableFuture<String> combined =
```

```

8      userFuture.thenCombine(
9          scoreFuture,
10         (name, score) -> name + " scored " + score
11     );

```

The combining function is a BiFunction:

```

1 BiFunction<T, U, V>

```

Conceptually:

```

1 First future produces T
2 Second future produces U
3           ↓
4       BiFunction<T, U, V>
5           ↓
6       Combined result V

```

The two futures can return different types.

```

1 CompletableFuture<String>
2 CompletableFuture<Integer>

```

The combined future may return a third type:

```

1 CompletableFuture<Result>

```

## Dependency behavior

The two stages can run independently:

```

1 Task 1 _____
2           |_____ combine results
3 Task 2 _____

```

The combination function runs only after both futures complete successfully.

As before:

```

1 thenCombine      → combining function may execute inline
2 thenCombineAsync → combining function is scheduled
  asynchronously

```

## 17. Sequential versus parallel composition

### Sequential dependency with thenCompose()

Use this when Task 2 requires Task 1's output:

```

1 findUser(userId)
2   .thenCompose(user -> findOrders(user));
1 Task 1 → Task 2

```

## Independent parallel tasks with thenCombine()

Use this when the tasks can run independently:

```

1 loadCustomer()
2   .thenCombine(loadExchangeRate(), this::createResponse);
1 Task 1 ┌
2         └→ combine
3 Task 2 ┌

```

This distinction is more important than simply thinking in terms of ordering.

---

## 18. Exception handling

A practical CompletableFuture pipeline also needs exception handling.

### exceptionally()

Provides a fallback value when an earlier stage fails:

```

1 CompletableFuture<String> future =
2   CompletableFuture.supplyAsync(() -> loadData())
3   .exceptionally(exception -> {
4       System.err.println(exception.getMessage());
5       return "fallback";
6   });

```

### handle()

Processes both success and failure:

```

1 CompletableFuture<String> future =
2   CompletableFuture.supplyAsync(() -> loadData())
3   .handle((result, exception) -> {
4       if (exception != null) {
5           return "fallback";
6       }
7
8       return result.toUpperCase();
9   });

```

## whenComplete()

Observes the result or exception without normally transforming it:

```
1 CompletableFuture<String> future =
2     CompletableFuture.supplyAsync(() -> loadData())
3     .whenComplete((result, exception) -> {
4         if (exception != null) {
5             System.err.println("Failed: " +
exception.getMessage());
6         } else {
7             System.out.println("Completed: " + result);
8         }
9     });
```

Simple distinction:

```
1 exceptionally → recover from failure
2 handle        → transform success or failure
3 whenComplete  → observe success or failure
```

## 19. Complete example with a controlled executor

```
1 import java.util.concurrent.CompletableFuture;
2 import java.util.concurrent.ExecutorService;
3 import java.util.concurrent.Executors;
4
5 public class CompletableFutureExample {
6
7     public static void main(String[] args) {
8         ExecutorService executor =
Executors.newFixedThreadPool(4);
9
10        try {
11            CompletableFuture<String> userFuture =
12                CompletableFuture.supplyAsync(
13                    () -> loadUser(),
14                    executor
15                );
16
17            CompletableFuture<Integer> scoreFuture =
18                CompletableFuture.supplyAsync(
```

```

19         () -> loadScore(),
20         executor
21     );
22
23     CompletableFuture<String> resultFuture =
24         userFuture
25             .thenCombineAsync(
26                 scoreFuture,
27                 (user, score) -> user + " scored " +
28                 score,
29                 executor
30             )
31             .thenApply(message ->
32                 message.toUpperCase())
33             .exceptionally(exception -> {
34                 System.err.println(
35                     "Pipeline failed: " +
36                     exception.getMessage()
37                 );
38                 return "RESULT UNAVAILABLE";
39             });
40
41     String result = resultFuture.join();
42     System.out.println(result);
43 } finally {
44     executor.shutdown();
45 }
46
47 private static String loadUser() {
48     return "Suhaib";
49 }
50
51 private static int loadScore() {
52     return 95;
53 }

```



## 20. get() versus join()

Both wait for completion.

### get()

```
1 String result = future.get();
```

It throws checked exceptions:

```
1 InterruptedException
2 ExecutionException
```

### join()

```
1 String result = future.join();
```

It throws an unchecked:

```
1 CompletionException
```

A common practical guideline is:

- Use get() when working with the general Future API or when checked exception handling is desired.
- Use join() when working inside CompletableFuture pipelines or where an unchecked completion exception is more convenient.

Both methods block the calling thread.

---

## 21. Quick method selection guide

```
1 Need to start a task with no result?
2     → runAsync()
3
4 Need to start a task that returns a result?
5     → supplyAsync()
6
7 Need to synchronously transform a previous value?
8     → thenApply()
9
10 Need to start another async operation using the previous
    value?
11     → thenCompose()
12
13 Need to consume the previous value without returning a value?
```

```
14     → thenAccept()
15
16 Need to execute an action without using the previous value?
17     → thenRun()
18
19 Need to combine two independent async results?
20     → thenCombine()
21
22 Need to recover from an exception?
23     → exceptionally()
24
25 Need to process both success and failure?
26     → handle()
27
28 Need to observe completion without changing the result?
29     → whenComplete()
```

## Key corrections to your original understanding

### Technical accuracy

- FutureTask **manages the task's completion state**, not ThreadPoolExecutor based on the worker thread's state.
- CompletableFuture **implements** both Future and CompletionStage.
- thenApplyAsync() does not guarantee a new thread. It schedules work through an executor.
- thenApply() does not guarantee that it will always run on the same thread as the preceding stage.
- thenAccept() is not strictly a termination operation because another stage can still follow it.
- thenCompose() provides dependent async composition and flattens nested futures.
- thenCombine() combines independent results after both stages complete.

### Clarity

The key model is:

```
1 Future
2     → observe or wait for one computation
3
4 CompletableFuture
5     → construct, transform, combine, recover, and observe
```

## 6 asynchronous computations

The most useful method distinction is:

- 1 `thenApply` =  $T \rightarrow U$
- 2 `thenCompose` =  $T \rightarrow \text{CompletableFuture}\langle U \rangle$
- 3 `thenAccept` =  $T \rightarrow \text{void}$
- 4 `thenRun` =  $() \rightarrow \text{void}$
- 5 `thenCombine` =  $(T, U) \rightarrow V$